

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Гельм Даниил

Группа:

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР1: Реализация REST API на основе boilerplate.

Требовалось:

1. Выбрать подходящий boilerplate для разработки backend-приложения.
2. На основе результатов ДЗ1 и ДЗ2 реализовать модели, представления и контроллеры.
3. Продемонстрировать полностью работающее API согласно выбранному варианту предметной области.

Вариант проекта: сайт для поиска работы.

Ход работы

1. Выбор технологического стека

Для реализации лабораторной работы был выбран стек:

- Node.js;
- TypeScript;
- Express;
- TypeORM;
- PostgreSQL;
- JWT для аутентификации.

Выбор обоснован тем, что данные технологии рассматриваются в рамках курса, а спроектированная ранее модель базы данных (Д31) и OpenAPI-спецификация (Д32) напрямую ложатся на REST API с реляционным хранилищем.

2. Структура проекта

Проект размещен в каталоге `labs/lab1` и включает:

- `src/types.ts` — типы доменной модели;
- `src/entities.ts` — описание сущностей TypeORM;
- `src/data-source.ts` — подключение к базе данных;
- `src/auth.ts` — middleware авторизации и проверки ролей;
- `src/index.ts` — маршруты API и обработчики запросов;
- `docker-compose.yml` — запуск PostgreSQL;
- `.env.example` — пример переменных окружения.

3. Реализация моделей

На основе ERD из Д31 были реализованы сущности:

- `users` — пользователи системы;
- `companies` — компании работодателей;
- `resumes` — резюме соискателей;
- `skills` — справочник навыков;
- `resume_skills` — связь резюме и навыков;
- `vacancies` — вакансии;
- `vacancy_skills` — связь вакансий и навыков;
- `applications` — отклики на вакансии;

- favorite_vacancies — избранные вакансии.

Связи между сущностями сохранены в соответствии с проектом базы данных: пользователь владеет резюме и компаниями, компания публикует вакансии, соискатель отправляет отклики и добавляет вакансии в избранное.

4. Реализация REST API

Базовый префикс API: /api/v1.

Реализованы следующие группы эндпоинтов:

Аутентификация:

- POST /auth/register — регистрация пользователя;
- POST /auth/login — вход и получение JWT-токена;
- GET /users/me — получение данных текущего пользователя.

Компании:

- GET /companies — список компаний;
- POST /companies — создание компании (роль employer или admin).

Резюме:

- GET /resumes — список резюме текущего пользователя;
- POST /resumes — создание резюме (роль applicant или admin).

Вакансии:

- GET /vacancies — список вакансий с фильтрацией (location, salaryFrom, salaryTo, experienceRequired);
- POST /vacancies — создание вакансии (роль employer или admin).

Отклики:

- POST /vacancies/:vacancyId/applications — отклик на вакансию;
- GET /applications/my — список откликов текущего соискателя;
- PATCH /applications/:applicationId/status — изменение статуса отклика (роль employer или admin).

Избранное:

- GET /favorites — список избранных вакансий;
- POST /vacancies/:vacancyId/favorite — добавить вакансию в избранное;
- DELETE /vacancies/:vacancyId/favorite — удалить из избранного.

Навыки:

- GET /skills — справочник навыков;
- POST /skills — добавление навыка (роль admin).

Дополнительно реализован служебный эндпоинт GET /health для проверки работоспособности сервера.

5. Авторизация и обработка ошибок

Для защищенных маршрутов используется JWT (Bearer Token). В токене хранятся идентификатор пользователя и роль (applicant, employer, admin). Для ограничения доступа применяется проверка роли на уровне middleware.

Ошибки API возвращаются в едином формате с полями `code` и `message`. Используются коды ответа: 401 (не авторизован), 403 (нет прав), 404 (не найдено), 409 (конфликт), 422 (ошибка валидации), 500 (внутренняя ошибка).

Валидация входных данных выполняется с помощью библиотеки `zod`.

6. Запуск и проверка работоспособности

База данных PostgreSQL поднимается через `docker compose`. Параметры подключения задаются в файле `.env`.

Запуск приложения:

- `npm install`;
- `npm run dev` (режим разработки) или `npm run build && npm start`.

Проверка API выполнялась отправкой HTTP-запросов к работающему серверу. Были успешно проверены сценарии: регистрация и вход пользователей, создание резюме, создание компании и вакансии, получение списка вакансий, отправка отклика, добавление вакансии в избранное, изменение статуса отклика работодателем.

Таким образом, реализованное API покрывает основные бизнес-сценарии варианта «сайт для поиска работы» и соответствует проекту, выполненному в ДЗ1 и ДЗ2.

Вывод

В ходе выполнения лабораторной работы № 1 было реализовано backend-приложение для сайта поиска работы на стеке Express + TypeScript + TypeORM + PostgreSQL. Спроектированные ранее сущности базы данных перенесены в модели ORM, реализованы REST-эндпоинты с аутентификацией по JWT и разграничением прав по ролям. API протестировано на основных пользовательских сценариях и готово к дальнейшему использованию в рамках курса (в том числе для тестирования в Postman в ДЗ3).